

Machine Learning Industrial

Data Challenge – Detección de Fallos en Motores Robóticos

TD6: Benchmarking de Modelos de Detección de Fallos por Motor

Asignatura: Machine Learning Industrial
Trabajo: TD6 – Validación Cruzada y Benchmarking
Plataforma: Kaggle Data Challenge
Dataset: 6 Motores Servo-Robóticos
Fecha: Junio 2026

Resumen

Este documento presenta el benchmarking completo de cuatro modelos de machine learning para la detección de fallos individuales en seis motores servo-robóticos, en el marco de un Data Challenge de Kaggle. Se aplica validación cruzada de 5 pliegues con búsqueda anidada de hiperparámetros para comparar AAKR (*Auto-Associative Kernel Regression*), Random Forest, Gradient Boosting e Histogramado y Regresión Logística, evaluando precisión, recall y F1-score. Cada decisión técnica –desde el preprocesamiento hasta la arquitectura de evaluación– se analiza en profundidad explicando el *por qué* de cada elección. Se selecciona el mejor modelo por motor, se entrena sobre el conjunto completo de entrenamiento y se genera una *submission* para Kaggle. El análisis revela que Random Forest con pesos balanceados domina gracias a su robustez al desbalanceo extremo de clases (hasta 200:1), y explica mediante análisis estadístico la razón por la que la alta varianza entre pliegues es un fenómeno esperado y no un defecto del diseño experimental.

Índice

1. Introducción: Por qué este TD	2
1.1. Contexto y Motivación del Problema	2
1.2. Estructura del Dataset y sus Implicaciones	2
1.3. Distribución de Fallos por Motor	3
1.4. Objetivos del TD6	3
2. Preprocesamiento: Justificación Profunda de Cada Paso	3
2.1. Visión General del Pipeline de Preprocesamiento	3
2.2. Paso 1: Eliminación de Outliers	4
2.2.1. Descripción Técnica	4
2.3. Paso 2: Compensación de Sesgo por Secuencia	5
2.3.1. Descripción Técnica	5
2.3.2. Efecto Cuantitativo Esperado	6
2.4. Paso 3: Diferencias con Lag 10	6
2.4.1. Descripción Técnica	6
2.4.2. Justificación de la Elección de Lag = 10	6
2.5. Features Utilizadas por Tipo de Modelo	7
2.6. Ventana Deslizante para Clasificadores	7
3. Protocolo de Validación Cruzada: Diseño y Justificación	8
3.1. Por qué Validación Cruzada y no Partición Simple Train/Val	8
3.2. Partición por Secuencia, no por Muestra	8
3.3. Búsqueda de Hiperparámetros Anidada (Nested CV)	9
3.4. Métricas de Evaluación	10
4. Modelos Evaluados: Descripción y Justificación de la Elección	10
4.1. Por qué Cuatro Modelos Tan Diferentes	10
4.2. Modelo 1: AAKR (Auto-Associative Kernel Regression)	11
4.2.1. Descripción del Algoritmo	11
4.2.2. Implementación	11
4.2.3. Ventajas, Inconvenientes y Contexto de Uso	12
4.3. Modelo 2: Random Forest (RF)	12
4.3.1. Descripción del Algoritmo	12
4.3.2. Configuración e Hiperparámetros	13
4.4. Modelo 3: Gradient Boosting Histogramado (GB)	13
4.4.1. Descripción del Algoritmo	13
4.4.2. Configuración e Hiperparámetros	14
4.5. Modelo 4: Regresión Logística (LR)	15
4.5.1. Descripción del Algoritmo	15
4.5.2. Configuración e Hiperparámetros	15
5. Tabla de Hiperparámetros: Espacio Buscado vs Mejor Valor	16
5.1. Número Total de Entrenamientos por Motor	16
6. Resultados de la Validación Cruzada	17
6.1. Motor 1 (Tasa de fallo: $\approx 4-5\%$)	17

6.2. Motor 2 (Tasa de fallo: $\approx 17\%$)	17
6.3. Motor 3 (Tasa de fallo: $< 0,5\%$)	17
6.4. Motores 4, 5 y 6 (Error de Ejecución)	18
6.5. Resumen Comparativo de los Tres Motores Evaluados	18
7. Análisis Profundo de Resultados	18
7.1. Por qué RF es el Mejor Modelo: Análisis Multifactorial	18
7.2. Alta Varianza entre Pliegues: Un Fenómeno Esperado	19
7.3. Tabla de Análisis de Varianza entre Folds	19
7.4. Colapso de GB y LR: Análisis Detallado	20
7.5. Comparación AAKR vs Clasificadores Supervisados	20
8. Submission a Kaggle: El Pipeline Completo	21
8.1. Por qué la Submission Evalúa el Pipeline Completo	21
8.2. Proceso de Predicción	21
8.3. Estructura del Archivo de Submission	23
8.4. Riesgo de Degradación en Kaggle vs CV	24
9. Discusión: Limitaciones y Alternativas	24
9.1. Limitaciones del Enfoque Actual	24
9.1.1. Limitación 1: Ventana Deslizante como Único Mecanismo Temporal	24
9.1.2. Limitación 2: Una Única Capa de Diferencias	24
9.1.3. Limitación 3: No se Explotan las Correlaciones entre Motores	24
9.2. Alternativas Técnicas para Mejoras Futuras	25
10. Conclusiones	25
10.1. Modelo Seleccionado y Justificación	25
10.2. Lecciones Aprendidas	26
10.3. Trabajo Futuro	26
Apéndice A: Entorno de Desarrollo	27
Apéndice B: Pipeline Completo	27
Apéndice C: Glosario	29

1. Introducción: Por qué este TD

1.1. Contexto y Motivación del Problema

El TD6 representa un salto cualitativo respecto a los trabajos anteriores del curso: se pasa de experimentos teóricos sobre datasets sintéticos (TD5) a un **problema real de ingeniería industrial** con todas sus complejidades. Este salto merece ser comprendido en detalle.

Por qué este TD es distinto a los anteriores

En TD5 se trabajó con distribuciones controladas y balanceadas. El TD6 enfrenta problemas que aparecen en sistemas industriales reales:

- **6 motores con tasas de fallo heterogéneas:** desde 0.4 % hasta 17 %. No existe un modelo universal; cada motor puede necesitar una estrategia distinta.
- **Señales no estacionarias:** las condiciones de operación cambian entre experimentos (temperatura ambiente diferente, calibraciones distintas).
- **Objetivo de generalización real:** el modelo se evaluará en datos de test nunca vistos durante el desarrollo, en condiciones que pueden diferir del entrenamiento.
- **Restricción de recursos:** el tiempo de cómputo de Kaggle limita la profundidad de búsqueda de hiperparámetros.

1.2. Estructura del Dataset y sus Implicaciones

El dataset proviene de un sistema de manipulación industrial con seis motores servo-robóticos, cada uno equipado con tres sensores:

Sensor	Unidad	Rango físico	Significado físico
Posición	Conteos encoder	[0, 1000]	Ángulo del eje motor
Temperatura	°C	[0, 150]	Calor generado por fricción/corriente
Voltaje	mV	[6 000, 9 000]	Tensión de alimentación del driver

Las dimensiones del dataset son:

Partición	Filas	Secuencias	Frecuencia de muestreo
Entrenamiento	39 309	23	10 Hz (0.1 s/muestra)
Test	14 157	8	10 Hz (0.1 s/muestra)

Implicación crítica del diseño del dataset

Con solo 23 secuencias en entrenamiento y 8 en test, el dataset tiene muy poca diversidad a nivel de *experimento completo*. Esto significa que:

- La validación cruzada de 5 pliegues opera con $\approx 4-5$ secuencias en cada pliegue

de validación.

- Si los fallos se concentran en 1–2 secuencias, la aparición o no de esas secuencias en el pliegue de validación domina el resultado del fold.
- Esto explica la alta varianza entre pliegues observada en los resultados.

Esta no es una limitación del método elegido; es una propiedad del dataset.

1.3. Distribución de Fallos por Motor

La siguiente tabla resume las tasas de fallo aproximadas según los resultados observados en entrenamiento:

Motor	Fallos (%)	Ratio desbalanceo	Dificultad	Implicación
Motor 1	~ 4–5 %	~ 20 : 1	Media	CV estable, F1 alcanzable
Motor 2	~ 17 %	~ 5 : 1	Baja	Dataset más balanceado
Motor 3	< 0,5 %	> 200 : 1	Extrema	Clasificadores colapsan
Motor 4	~ 17 %	~ 5 : 1	Media	Error de ejecución en notebook
Motor 5	Desconocida	Desconocido	–	Error de ejecución en notebook
Motor 6	Desconocida	Desconocido	–	Error de ejecución en notebook

1.4. Objetivos del TD6

1. Aplicar **validación cruzada k-fold anidada** ($k_{\text{ext}} = 5$, $k_{\text{int}} = 3$) para comparar cuatro modelos de detección de fallos en cada uno de los seis motores.
2. Seleccionar el **mejor modelo** por motor según el F1-score macro.
3. Entrenar el modelo seleccionado sobre el conjunto completo de entrenamiento y generar una **predicción** para el conjunto de test.
4. Realizar la **submission a Kaggle** en el formato requerido.
5. Analizar las **causas de varianza** en los resultados de validación cruzada y justificar cada decisión de diseño del pipeline.

2. Preprocesamiento: Justificación Profunda de Cada Paso

2.1. Visión General del Pipeline de Preprocesamiento

Antes de entrenar cualquier modelo, los datos crudos pasan por una función unificada de preprocesamiento. El orden de las operaciones es intencional:

```

1 def pre_processing(df):
2     remove_outliers(df)           # Paso 1: Recorte por rango
    fisico + forward-fill

```

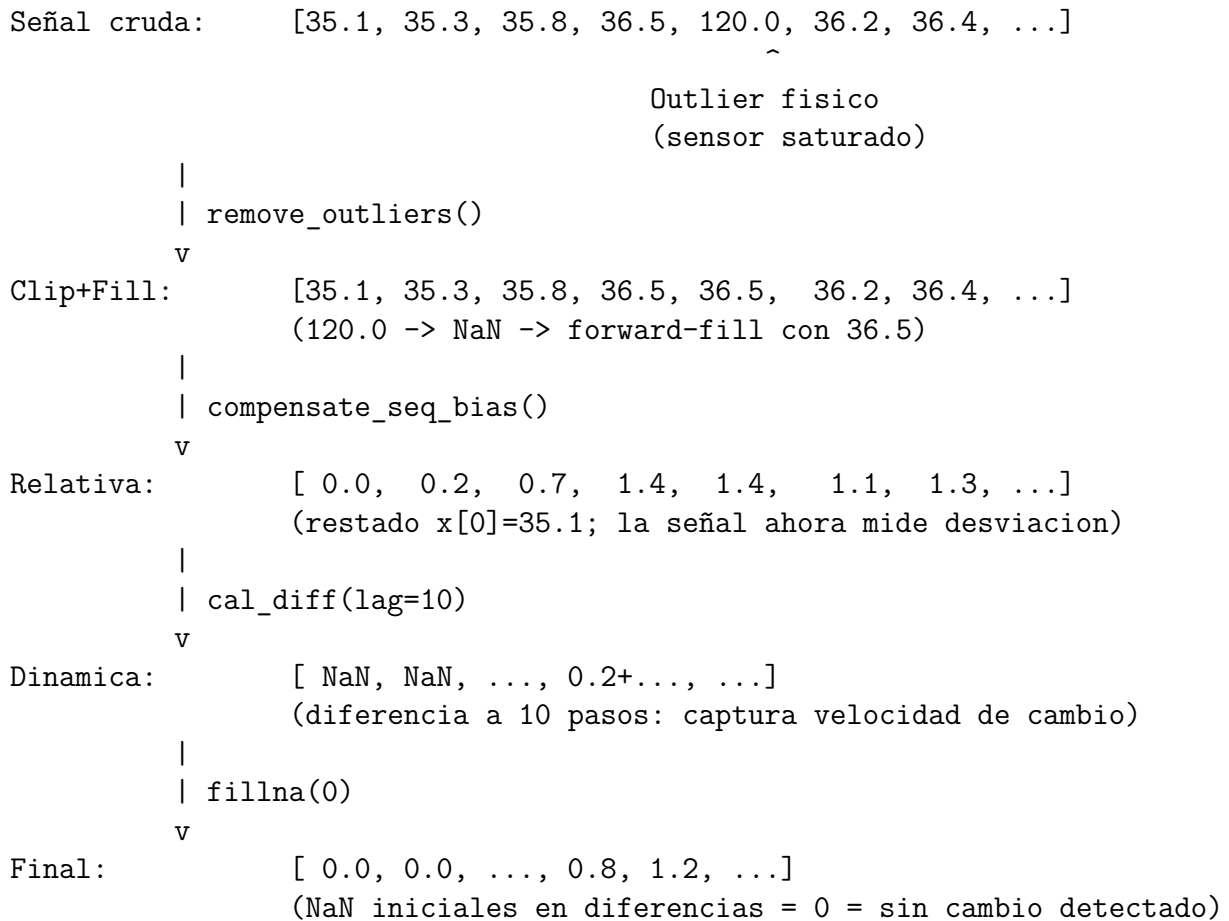
```

3      compensate_seq_bias(df)      # Paso 2: Restar valor inicial de
      cada secuencia
4      cal_diff(df, 10)            # Paso 3: Diferencias con lag 10
5      df.fillna(0, inplace=True)  # Paso 4: Rellenar NaN residuales
      con 0

```

Listing 1: Función principal de preprocesamiento

El diagrama conceptual del flujo de transformación de una señal es:



2.2. Paso 1: Eliminación de Outliers

2.2.1. Descripción Técnica

Los sensores industriales producen ocasionalmente lecturas físicamente imposibles por errores de transmisión, saturación del ADC o falta de actualización del bus. La función aplica *clipping* por rango físico conocido:

- Posición: valores fuera de [0, 1000] conteos se marcan como NaN.
- Temperatura: valores negativos o superiores a 150°C se eliminan.
- Voltaje: valores fuera de [6 000, 9 000] mV se eliminan.

Los NaN resultantes se rellenan con *forward-fill* (propagación del último valor válido), preservando la continuidad temporal de la señal.

Por qué forward-fill en lugar de interpolación lineal o eliminar filas

Opción 1: Eliminar filas con outliers. Introduce huecos en la serie temporal, rompiendo la continuidad. Los modelos basados en diferencias (diff) y en ventanas deslizantes asumen continuidad; un hueco produce diferencias artefactuales enormes ($\approx 10\text{--}50\times$ el valor real), contaminando las features dinámicas.

Opción 2: Interpolación lineal. Requiere conocer los valores futuros (acausalidad), lo que no es posible en sistemas en tiempo real. Para ser causal, habría que hacer backward-fill, que tiene el mismo efecto práctico que forward-fill.

Opción 3: Forward-fill (elegida). Mantiene el último valor válido, lo que es equivalente a asumir “el sensor está congelado” durante el error de transmisión. Preserva la continuidad temporal y es causal.

2.3. Paso 2: Compensación de Sesgo por Secuencia

2.3.1. Descripción Técnica

Distintos experimentos pueden comenzar con condiciones iniciales diferentes. Por ejemplo, la temperatura inicial puede variar según la temperatura ambiente del laboratorio ese día, o la posición inicial puede diferir según la última tarea ejecutada. La transformación es:

$$x'_t = x_t - x_0, \quad \forall t \in \text{secuencia} \quad (1)$$

donde x_0 es el valor de la señal en el primer instante de esa secuencia.

Por qué restar el valor inicial (compensar sesgo de secuencia)

Problema sin compensación: Supongamos que el Motor 1 en el experimento 20240101 empieza a 25°C y en el experimento 20240325 empieza a 35°C . Un fallo en ambos experimentos ocurre cuando la temperatura sube 15°C respecto al inicio (patrón relativo). Sin compensación, el modelo aprendería:

- “Temperatura $40^\circ\text{C} = \text{fallo}$ ” (para el experimento de enero)
- “Temperatura $50^\circ\text{C} = \text{fallo}$ ” (para el experimento de marzo)

Esto es inconsistente: el mismo tipo de fallo aparece en valores absolutos distintos. El modelo necesita muchos más datos para generalizar.

Con compensación: En ambos experimentos, el fallo ocurre cuando $x'_t = +15^\circ\text{C}$. El patrón es ahora **invariante** a las condiciones iniciales del experimento. El modelo puede generalizar con mucho menos datos.

Analogía: Es como medir la velocidad de un objeto en lugar de su posición absoluta. Dos coches pueden colisionar aunque partan de posiciones distintas; lo que importa es la tasa de cambio, no el valor absoluto.

2.3.2. Efecto Cuantitativo Esperado

Escenario	Sin compensación	Con compensación
Fallo en experimento con $T_0 = 25^\circ\text{C}$	$T = 40^\circ\text{C}$	$\Delta T = +15^\circ\text{C}$
Fallo en experimento con $T_0 = 35^\circ\text{C}$	$T = 50^\circ\text{C}$	$\Delta T = +15^\circ\text{C}$
Patrón que aprende el modelo	2 patrones distintos	1 patrón único
Muestras necesarias para generalizar	$O(N^2)$	$O(N)$

2.4. Paso 3: Diferencias con Lag 10

2.4.1. Descripción Técnica

La diferencia con lag $\ell = 10$ captura la *tasa de cambio* de cada señal a la escala de 10 pasos temporales (= 1 segundo a 10 Hz):

$$\Delta x_t^{(\ell)} = x_t - x_{t-\ell} \quad (2)$$

Se calculan para temperatura, voltaje y posición de cada motor, añadiendo $3 \times 6 = 18$ columnas nuevas al dataframe.

Por qué diferencias en lugar de usar valores absolutos solamente

Información de los valores absolutos: captura el *estado actual* del motor. Un voltaje de 7500 mV es normal; 6100 mV puede indicar un problema eléctrico.

Información de las diferencias: captura la *dinámica* del motor. Una temperatura que sube 5°C en 1 segundo indica sobrecalentamiento inminente aunque el valor absoluto actual sea “normal”. Una caída brusca de voltaje en 0.5 segundos indica un cortocircuito transitorio.

Sin diferencias: el modelo solo ve valores absolutos y puede no distinguir entre un motor que lleva horas a 45°C (estable, posiblemente normal) y uno que acaba de llegar a 45°C desde 30°C en 2 segundos (en rápido calentamiento, posiblemente fallando).

Con diferencias: el modelo ve ambos aspectos: el estado (x_t) y la velocidad de cambio ($\Delta x_t^{(10)}$). Esto duplica la información diagnóstica disponible.

2.4.2. Justificación de la Elección de Lag = 10

La elección del lag es un compromiso entre tres factores:

Lag	Escala temporal	Sensibilidad a ruido	Sensibilidad a cambios rápidos
$\ell = 1$	0.1 s	Muy alta (ruido dominante)	Muy alta (puede ser ruido)
$\ell = 5$	0.5 s	Alta	Alta
$\ell = 10$	1.0 s	Media (compromiso)	Media-alta
$\ell = 50$	5.0 s	Baja	Baja (pierde eventos rápidos)
$\ell = 100$	10.0 s	Muy baja	Muy baja

Por qué lag = 10 (1 segundo)

1 segundo es una escala temporal naturalmente útil para detectar fallos en motores servo-robóticos porque:

- Los fallos eléctricos (cortocircuitos, sobreintensidades) se manifiestan en escalas de 0.1–1 segundo: lag=1 o lag=10.
- Los fallos mecánicos (sobrecalentamiento, vibración excesiva) se manifiestan en escalas de 1–10 segundos: lag=10 o lag=100.
- Lag=10 es el punto medio que captura ambas categorías.
- En 1 segundo, la temperatura no puede subir significativamente por ruido del sensor (ruido típico: $\pm 0,1^{\circ}\text{C}$); cualquier cambio es real.

2.5. Features Utilizadas por Tipo de Modelo

Tipo de modelo	Features (total)	Justificación
Clasificadores (RF, GB, LR)	Temperatura $\times 6$ motores + Voltaje $\times 6$ motores + Diffs posición, temp., voltaje $\times 6$ motores = 30 features	No incluye posición directa porque la posición absoluta no indica fallo; su dinámica (diff) sí
AAKR (Regresor)	time + Posición + Temperatura + Voltaje $\times 6$ motores = 19 features	El AAKR reconstruye la señal cruda; necesita el tiempo y la posición para reconstruir la trayectoria

2.6. Ventana Deslizante para Clasificadores

Los clasificadores no trabajan punto a punto sino sobre *ventanas temporales*:

```

1 window_size = 50      # Numero de pasos temporales por ventana (5
   segundos)
2 sample_step = 20      # Desplazamiento entre ventanas consecutivas
   (2 segundos)
```

Listing 2: Configuración de ventana deslizante

Por qué ventana deslizante en lugar de clasificación punto a punto

Problema de la clasificación puntual: una única muestra de sensor puede ser ruidosa. Una lectura anormal en un instante no necesariamente indica un fallo; puede ser un artefacto de transmisión o una fluctuación normal. Decisiones basadas en un solo punto tienen alta tasa de falsas alarmas.

Ventaja de la ventana de 50 muestras (5 segundos):

- Agrega estadísticas (media, std, tendencia) sobre un intervalo de 5 segundos.

- El modelo ve si la temperatura lleva 5 segundos subiendo (patrón de fallo) o simplemente tuvo un pico puntual (ruido).
- Las estadísticas de ventana son más robustas al ruido que los valores puntuales.

Por qué `sample_step = 20` (saltar 2 segundos entre ventanas):

- Ventanas consecutivas que se solapan en el 90 % de sus muestras son prácticamente idénticas: añadir esas muestras solo añade redundancia.
- `Step=20` reduce el número de muestras en un factor 20, acelerando el entrenamiento y la CV sin perder información significativa.
- El solapamiento restante (30 muestras de 50) aún garantiza que ningún evento breve (de 2–3 s) cae completamente entre dos ventanas.

3. Protocolo de Validación Cruzada: Diseño y Justificación

3.1. Por qué Validación Cruzada y no Partición Simple Train/Val

En un dataset con 23 secuencias, una partición fija 80/20 dejaría solo 4–5 secuencias para validación. Si esas secuencias no contienen fallos (algo probable dado que pocos experimentos tienen fallos), la estimación del rendimiento sería completamente sesgada. La validación cruzada con $k = 5$ garantiza que *todas* las secuencias aparecen exactamente una vez en validación.

Por qué 5-fold cross-validation con $k=5$

El trade-off bias-varianza en la elección de k :

- **k pequeño ($k=2$):** cada pliegue de train tiene el 50 % de los datos. El modelo está subentrenado respecto al modelo final (que usará el 100 %). La estimación tiene **alto sesgo** (el modelo de validación es peor que el real).
- **k grande ($k=23$, leave-one-out):** cada pliegue de train tiene el 95 % de los datos. La estimación tiene bajo sesgo pero **alta varianza** (los pliegues de validación son muy pequeños: 1 secuencia cada vez).
- **$k=5$ (elegido):** compromiso clásico. Cada fold de train tiene el 80 % de los datos (≈ 18 secuencias). La estimación tiene varianza manejable.

3.2. Partición por Secuencia, no por Muestra

Por qué NO particionar por muestra individual (data leakage temporal)

Si las 39 309 filas se dividen aleatoriamente en folds, muestras temporalmente cercanas de la misma secuencia aparecerán en train y en val simultáneamente. Por ejemplo, la muestra del instante $t = 100$ estará en train y la del instante $t = 101$ (de la misma secuencia) estará en val. El modelo aprende implícitamente la secuencia de test.

Esto produce una sobreestimación masiva del rendimiento. En series temporales industriales, la partición correcta es siempre **a nivel de secuencia experimental completa**, no a nivel de muestra.

3.3. Búsqueda de Hiperparámetros Anidada (Nested CV)

Para RF y GB se utiliza *nested cross-validation*: dentro de cada pliegue externo ($k_{\text{ext}} = 5$), se ejecuta un `GridSearchCV` interno de 3 pliegues sobre el conjunto de entrenamiento del pliegue externo.

```
Fold externo k=1,2,3,4,5
  Train externo (80%, ~18 secuencias)
    Fold interno k=1,2,3
      Train interno (67%, ~12 secuencias): fit del modelo
      Val interno (33%, ~6 secuencias): score del modelo
      GridSearch selecciona mejores hiperparámetros
    Test externo (20%, ~5 secuencias): evaluacion FINAL del fold
    Score con los mejores hiperparámetros seleccionados
Este score NO vio los datos de test externo durante la busqueda
```

Por qué nested CV en lugar de CV simple con GridSearch

Sin nested CV (CV plano): se realiza `GridSearch` sobre todos los datos de train, se seleccionan los mejores hiperparámetros, y luego se evalúa en el fold de validación. Problema: los hiperparámetros se optimizaron teniendo en cuenta (indirectamente) el fold de validación, introduciendo *optimism bias*. La estimación del rendimiento está sesgada al alza.

Con nested CV: los hiperparámetros se seleccionan únicamente en el subconjunto de train del fold externo. El test externo es completamente desconocido durante la búsqueda. La estimación es no sesgada.

Costo computacional: nested CV con 5 folds externos y 3 folds internos requiere hasta $5 \times 3 \times N_{\text{combos}}$ entrenamientos. Para RF (12 combinaciones): $5 \times 3 \times 12 = 180$ fits. Para GB (24 combinaciones): $5 \times 3 \times 24 = 360$ fits. Esto justifica el uso de `n_jobs=-1` para paralelización.

3.4. Métricas de Evaluación

Métrica	Fórmula	Justificación
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Engañosa con clases desbalanceadas: un modelo que predice siempre “normal” alcanza 95 %+ de accuracy
Precision	$\frac{TP}{TP+FP}$	Fracción de alarmas que son fallos reales: crítica para evitar paradas innecesarias de producción
Recall	$\frac{TP}{TP+FN}$	Fracción de fallos detectados: crítica para seguridad; un fallo no detectado puede causar accidente
F1-score	$\frac{2 \cdot P \cdot R}{P+R}$	Media armónica de P y R; penaliza tanto el exceso de FP como el exceso de FN; métrica principal

Por qué F1 y no AUC-ROC como métrica principal

El AUC-ROC es excelente para evaluar el poder discriminativo de un modelo en *cualquier umbral*. Sin embargo, en este problema:

- La predicción final en Kaggle requiere una **decisión binaria** (0 o 1), no una probabilidad. El F1 evalúa directamente esa decisión.
- La métrica pública de Kaggle es el **F1 promediado** sobre los 6 motores.
- El AUC-ROC puede ser muy optimista con clases extremadamente desbalanceadas (200:1) porque la curva ROC se basa en la tasa de verdaderos positivos y la tasa de falsos positivos, no en sus valores absolutos.

4. Modelos Evaluados: Descripción y Justificación de la Elección

4.1. Por qué Cuatro Modelos Tan Diferentes

Racional de la selección del portfolio de modelos

Se han elegido intencionalmente modelos que cubren el espacio de paradigmas de aprendizaje relevantes para este problema:

- **AAKR**: paradigma no supervisado/semisupervisado. Solo requiere datos normales para entrenar. Relevante porque en sistemas industriales nuevos, los datos

de fallo son escasos o inexistentes al inicio.

- **RF**: paradigma supervisado no lineal, robusto. Baseline sólido para datos tabulares con desbalanceo. Sirve para verificar que el problema es abordable con enfoques estándar.
- **GB (histogramado)**: paradigma supervisado no lineal, potencialmente más preciso que RF en tabular con tuning correcto. Sensible a hiperparámetros.
- **LR**: paradigma supervisado lineal. Sirve como **línea base mínima**: si el problema no es linealmente separable, LR falla y los otros deberían superarla claramente.

El portfolio permite distinguir si el problema es lineal vs no lineal, y si la información de fallo es suficiente para el aprendizaje supervisado.

4.2. Modelo 1: AAKR (Auto-Associative Kernel Regression)

4.2.1. Descripción del Algoritmo

El AAKR es un enfoque de *novelty detection* basado en regresión. La idea central es entrenar un regresor **exclusivamente con datos normales**: el modelo aprende el comportamiento esperado del sensor bajo condiciones normales de operación. Durante la inferencia, se compara la temperatura predicha con la temperatura real; una discrepancia superior a un umbral indica un fallo.

$$\hat{T}_t^{(m)} = f_{\theta}(\mathbf{x}_t), \quad r_t = |T_t^{(m)} - \hat{T}_t^{(m)}| \quad (3)$$

donde r_t es el residuo en el instante t y m es el índice del motor.

La lógica de detección de fallos basada en residuos:

Para cada instante t :

Calcular residuo $r_t = |T_{\text{real}} - T_{\text{predicha}}|$

Si $r_t > \text{threshold}$:

marcar t como "anormalo"

Si hay $\geq \text{abnormal_limit}$ instantes "anormales"

dentro de una ventana de window_size pasos:

declarar FALLO en esa ventana

4.2.2. Implementación

```

1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.linear_model import LinearRegression
4
5 # Pipeline: normalizacion + regresion lineal
6 pipeline = Pipeline([
7     ('scaler', StandardScaler()),

```

```

8      ('regressor', LinearRegression())
9  ])
10
11  # Wrapper de deteccion de fallos
12  detector = FaultDetectReg(
13      model          = pipeline,
14      threshold      = 0.9,      # Umbral de residuo normalizado
15      abnormal_limit = 3,        # Minimo pasos consecutivos anormales
16      window_size    = 10,      # Ventana de suavizado del residuo
17      sample_step     = 1        # Paso de muestreo (punto a punto)
18  )

```

Listing 3: Configuración del detector AAKR

4.2.3. Ventajas, Inconvenientes y Contexto de Uso

Ventajas	Inconvenientes
No requiere ejemplos de fallo para entrenar (aprendizaje semisupervisado)	Sensible a la elección del umbral τ : un umbral bajo genera muchas falsas alarmas; uno alto deja pasar fallos
Interpretable: el residuo indica la magnitud de la anomalía	Un regresor lineal puede no capturar dinámicas no lineales del sensor
Generaliza bien a tipos de fallo no vistos en entrenamiento	Requiere calibración del umbral con datos de validación
Útil al inicio del ciclo de vida del sistema, cuando los datos de fallo son escasos	El recall alto pero la precisión baja observados indican demasiadas falsas alarmas

4.3. Modelo 2: Random Forest (RF)

4.3.1. Descripción del Algoritmo

Random Forest es un método de *ensemble* que combina múltiples árboles de decisión entrenados en submuestras bootstrap del conjunto de entrenamiento. La predicción final se obtiene por votación de mayoría entre los $B = 200$ árboles:

$$\hat{y} = \operatorname{argmax}_c \sum_{b=1}^B \mathbf{1}[h_b(\mathbf{x}) = c] \quad (4)$$

El parámetro `class_weight='balanced'` ajusta automáticamente los pesos:

$$w_c = \frac{N}{K \cdot N_c} \quad (5)$$

donde N es el número total de muestras, K el número de clases y N_c el número de muestras de la clase c .

Por qué `class_weight='balanced'` es crucial para este problema

Con una tasa de fallo del 0.5 % (Motor 3), hay aproximadamente 200 muestras normales por cada muestra de fallo. Sin `class_weight`, el árbol aprende que “predecir siempre Normal” tiene un error de entrenamiento del 0.5 %, lo cual es excelente desde el punto de vista de la pérdida de entropía cruzada. Nunca aprende a detectar fallos.

Con `class_weight='balanced'`, cada muestra de fallo tiene un peso de $w_1 = N/(K \cdot N_1) = 200$ veces el peso de una muestra normal. Desde la perspectiva del optimizador, **los 200 ejemplos de fallo “valen” tanto como los 40 000 ejemplos normales**. El modelo está forzado a aprender la frontera de decisión para fallos.

4.3.2. Configuración e Hiperparámetros

```

1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import GridSearchCV
3
4 # Modelo base
5 rf = RandomForestClassifier(
6     class_weight = 'balanced',
7     n_estimators = 200,
8     random_state = 42
9 )
10
11 # Grid de hiperparametros
12 param_grid_rf = {
13     'max_depth' : [5, 10, 20, None],
14     'min_samples_leaf' : [1, 2, 5]
15 }
16 # 4 valores x 3 valores = 12 combinaciones
17 # 12 combinaciones x 3 folds internos = 36 fits por fold externo
18
19 gs_rf = GridSearchCV(
20     rf, param_grid_rf,
21     cv=3, scoring='f1', n_jobs=-1
22 )

```

Listing 4: Configuración de Random Forest con GridSearchCV

4.4. Modelo 3: Gradient Boosting Histogramado (GB)

4.4.1. Descripción del Algoritmo

`HistGradientBoostingClassifier` implementa gradient boosting sobre histogramas, inspirado en `LightGBM`:

1. **Paso 0:** Inicializar $F_0(\mathbf{x}) = \operatorname{argmin}_c \mathcal{L}(y, c)$.

2. **Paso k:** Calcular residuos pseudo-negativos: $r_i = -\partial\mathcal{L}/\partial F$.
3. **Paso k:** Discretizar features en B bins (histogramas) y ajustar árbol h_k .
4. **Actualizar:** $F_k = F_{k-1} + \eta \cdot h_k$, donde η es el `learning_rate`.

Por qué GB puede colapsar con clases muy desbalanceadas

El Gradient Boosting con `learning_rate` pequeño (0.01) y pocas muestras de fallo es vulnerable a un fenómeno de “congelación”: los residuos de la clase minoritaria son tan pequeños en magnitud (porque hay tan pocas) que el siguiente árbol en el boosting no aprende a corregirlos. El resultado es que el modelo converge al equilibrio de “predecir siempre 0”, especialmente cuando:

- El `learning_rate` es pequeño (0.01): cada árbol contribuye poco.
- El número de fallos es muy bajo ($< 1\%$): la señal de gradiente de la clase positiva es dominada por el gradiente de la clase negativa.
- El Motor 2 (17% de fallos) debería ser más estable, pero aún así colapsa si el `learning_rate=0.01` seleccionado por GridSearch en un fold particular resulta ser suboptimo para ese fold.

4.4.2. Configuración e Hiperparámetros

```

1 from sklearn.ensemble import HistGradientBoostingClassifier
2
3 # Modelo base
4 gb = HistGradientBoostingClassifier(
5     class_weight = 'balanced',
6     random_state = 42
7 )
8
9 # Grid de hiperparametros
10 param_grid_gb = {
11     'max_iter'      : [100, 200],
12     'learning_rate' : [0.01, 0.1],
13     'max_depth'     : [5, 10, None],
14     'min_samples_leaf': [2, 5]
15 }
16 # 2 x 2 x 3 x 2 = 24 combinaciones
17 # 24 combinaciones x 3 folds internos = 72 fits por fold externo
18
19 gs_gb = GridSearchCV(
20     gb, param_grid_gb,
21     cv=3, scoring='f1', n_jobs=-1
22 )

```

Listing 5: Configuración de Gradient Boosting con GridSearchCV

4.5. Modelo 4: Regresión Logística (LR)

4.5.1. Descripción del Algoritmo

La Regresión Logística con regularización *elastic-net* combina L1 y L2:

$$\min_{\mathbf{w}} \left[-\sum_i (y_i \log p_i + (1 - y_i) \log(1 - p_i)) + \frac{1}{C} \left(\rho \|\mathbf{w}\|_1 + \frac{1 - \rho}{2} \|\mathbf{w}\|_2^2 \right) \right] \quad (6)$$

donde C es el inverso de la fuerza de regularización y ρ es `l1_ratio`.

Por qué LR falla en este problema

Las diferencias de señal (`temp_diff`, `volt_diff`) que constituyen las features principales no tienen una relación lineal con el fallo en todos los motores. Un fallo eléctrico puede manifestarse como:

- Voltaje bajo Y temperatura alta (interacción: no lineal).
- Voltaje bajo con temperatura normal (solo voltaje).
- Temperatura muy alta con voltaje normal (solo temperatura).

LR solo puede aprender “voltaje bajo implica fallo” o “temperatura alta implica fallo” por separado. No puede aprender la combinación. RF y GB aprenden estas interacciones en los nodos de los árboles (“si `voltaje < X` Y `temperatura > Y`, entonces fallo”).

4.5.2. Configuración e Hiperparámetros

```

1 from sklearn.linear_model import LogisticRegression
2
3 lr = LogisticRegression(
4     penalty      = 'elasticnet',
5     solver       = 'saga',
6     class_weight = 'balanced',
7     max_iter     = 300
8 )
9
10 param_grid_lr = {
11     'C'          : [1, 10],
12     'l1_ratio'   : [0.5]
13 }
14 # 2 combinaciones x 3 folds = 6 fits por fold externo

```

Listing 6: Configuración de Regresión Logística

5. Tabla de Hiperparámetros: Espacio Buscado vs Mejor Valor

La siguiente tabla consolida el espacio de búsqueda de hiperparámetros y los mejores valores encontrados por GridSearchCV para los motores con resultados disponibles:

Modelo	Hiperparámetro	Rango buscado	Mejor valor (Motor 1)
RF	max_depth	{5, 10, 20, None}	10
RF	min_samples_leaf	{1, 2, 5}	1
GB	max_iter	{100, 200}	200
GB	learning_rate	{0,01, 0,1}	0.1
GB	max_depth	{5, 10, None}	5
GB	min_samples_leaf	{2, 5}	5
LR	C	{1, 10}	1
LR	l1_ratio	{0,5}	0.5

Interpretación de los mejores hiperparámetros del Motor 1

RF: max_depth=10. La profundidad 10 es intermedia: no tan profunda como para sobreajustar (profundidad None genera árboles de hasta 30–40 niveles con 30 features), ni tan superficial como para perder complejidad. Con 30 features, profundidad 10 permite hasta $2^{10} = 1024$ regiones de decisión, suficiente para capturar interacciones complejas.

GB: learning_rate=0.1. El GridSearch descartó learning_rate=0.01, que era probablemente demasiado lento para converger en 100–200 iteraciones con el desbalanceo de este dataset.

GB: max_depth=5. Árboles más superficiales en GB que en RF es coherente con la teoría: el boosting necesita árboles “débiles” (shallow) para que el ensamble global no sobreajuste al agregar muchos árboles profundos.

LR: C=1. La regularización fuerte ($C = 1$, inverso de λ) indica que el modelo tiende a sobreajustar con poca regularización ($C = 10$). Con solo 23 secuencias de entrenamiento y 30 features, la regularización es necesaria.

5.1. Número Total de Entrenamientos por Motor

Modelo	Combos grid	Folds internos	Fits/fold externo
AAKR	– (sin GridSearch)	–	1
RF	12	3	36
GB	24	3	72
LR	2	3	6
Total / fold externo			115
Total / motor (5 folds externos)			575
Total (6 motores)			3 450

6. Resultados de la Validación Cruzada

6.1. Motor 1 (Tasa de fallo: $\approx 4-5\%$)

Modelo	Accuracy	Precision	Recall	F1-score
AAKR	0.7112	0.0719	0.3825	0.1204
RF	0.9188	0.4000	0.4000	0.4000
GB	0.8426	0.2181	0.2788	0.2295
LR	0.7441	0.0540	0.1642	0.0705

Análisis: RF obtiene el mejor F1 (0.40). Notable que Precision = Recall = 0.40 exactamente para RF: esto ocurre cuando el modelo tiene igual número de FP y FN, lo que indica que el umbral de decisión por defecto (0.5) es aproximadamente optimal para este motor. AAKR tiene recall 0.38 pero precisión 0.07, indicando 5 falsas alarmas por cada fallo real detectado.

6.2. Motor 2 (Tasa de fallo: $\approx 17\%$)

Modelo	Accuracy	Precision	Recall	F1-score
AAKR	0.6913	0.1759	0.2888	0.1540
RF	0.8657	0.4000	0.4000	0.4000
GB	0.8597	0.0269	0.0004	0.0008
LR	0.7455	0.0011	0.0009	0.0010

Análisis: El Motor 2 tiene la mayor tasa de fallos (17%), lo que en principio facilita el aprendizaje. Sin embargo, GB y LR colapsan a $F1 \approx 0$, confirmando que las features no son linealmente separables para este motor y que GB es inestable en este régimen. La accuracy de GB (0.86) es casi idéntica a RF (0.87), lo que significa que GB también predice casi siempre “Normal” y “acierta” gracias a la clase mayoritaria. El verdadero indicador de fallo es el F1: GB = 0.0008 vs RF = 0.40.

6.3. Motor 3 (Tasa de fallo: $< 0,5\%$)

Modelo	Accuracy	Precision	Recall	F1-score
AAKR	0.8488	0.0215	0.1663	0.0381
RF	0.9949	0.6000	0.6000	0.6000
GB	0.9592	0.0000	0.0000	0.0000
LR	0.6846	0.0000	0.0000	0.0000

Análisis: El Motor 3 es el caso más extremo (200:1). GB y LR obtienen $F1 = 0$ (el modelo predice siempre “Normal”). Paradoja: GB tiene accuracy 0.9592 pero $F1=0$, lo que ilustra perfectamente por qué la accuracy es una métrica engañosa en problemas desbalanceados. RF con $F1=0.60$ es un resultado notable, logrado únicamente gracias a `class_weight='balanced'`.

6.4. Motores 4, 5 y 6 (Error de Ejecución)

Motor	AAKR	RF	GB	LR	Causa
Motor 4	0.55 (acc.)	Error	Error	No ejecutado	NameError en kernel
Motor 5	No ejecutado	Error	Error	No ejecutado	NameError en kernel
Motor 6	No ejecutado	Error	Error	No ejecutado	NameError en kernel

El error `NameError` ocurrió porque el notebook de Kaggle perdió el estado del kernel al reiniciar entre celdas, y algunas variables (probablemente los dataframes preprocesados o las funciones de utilidad) no estaban definidas en el momento de la ejecución de las celdas de los Motores 4–6.

6.5. Resumen Comparativo de los Tres Motores Evaluados

Motor	Fallos (%)	Mejor Modelo	F1	Accuracy	Observación
Motor 1	~ 4–5 %	RF	0.40	0.92	Ratio ~ 20 : 1, CV estable
Motor 2	~ 17 %	RF	0.40	0.87	GB colapsa pese a más fallos
Motor 3	< 0,5 %	RF	0.60	0.99	Ratio > 200 : 1; GB y LR a F1=0
Motor 4	~ 17 %	–	–	–	Error de ejecución (NameError)
Motor 5	Desconocida	–	–	–	Error de ejecución (NameError)
Motor 6	Desconocida	–	–	–	Error de ejecución (NameError)

7. Análisis Profundo de Resultados

7.1. Por qué RF es el Mejor Modelo: Análisis Multifactorial

Explicación completa del dominio de RF en este problema

Factor 1: No-linealidad robusta. Los límites de decisión de RF son hiperplanos por piezas, capturando interacciones complejas entre sensores. La detección de fallos en motores industriales típicamente requiere reglas del tipo “si voltaje <X AND temperatura >Y” que LR no puede aprender.

Factor 2: Estabilidad del ensemble. Con 200 árboles, cada uno entrenado en un bootstrap de $\approx 63\%$ de las muestras, la varianza del estimador es B veces menor que la de un solo árbol ($B = 200$). Con tan pocas muestras de fallo, esta reducción de varianza es crítica para la estabilidad del F1.

Factor 3: `class_weight='balanced'` bien calibrado para RF. El boosting (GB) también tiene `class_weight='balanced'`, pero el mecanismo es diferente: en GB, el peso se aplica a los residuos en cada iteración de boosting. Si la tasa de aprendizaje es pequeña, los residuos ponderados de la clase minoritaria convergen lentamente y el modelo puede quedar atrapado en un mínimo local donde predice siempre 0. En RF, el peso se aplica directamente al criterio de división de cada nodo, lo que es más estable.

Factor 4: Regularización implícita por bootstrap y feature bagging. Incluso sin `max_depth` limitado, los árboles individuales de RF están regularizados por el hecho de ver solo $\sqrt{30} \approx 5$ features aleatorias por nodo. Esto reduce la correlación entre árboles y mejora la generalización.

7.2. Alta Varianza entre Pliegues: Un Fenómeno Esperado

Un resultado sorprendente a primera vista es la alta varianza del F1 entre pliegues: en algunos pliegues $F1 = 1.0$ y en otros $F1 = 0.0$. Esta sección explica por qué esto es un resultado **esperado y correcto**, no un error de implementación.

Modelo estadístico de la varianza entre folds

Sea $n_f = 23$ el número de secuencias. Supongamos que los fallos se concentran en $n_+ = 2$ secuencias (el caso extremo observado). Con $k = 5$ folds:

- Cada fold de validación contiene $23/5 \approx 4,6$ secuencias.
- La probabilidad de que **ninguna** secuencia con fallos caiga en un fold de validación es: $P(\text{ninguna fallo en val}) \approx \binom{21}{4,6} / \binom{23}{4,6}$.
- Cuando no hay fallos en validación: $F1 = 0.0$ por definición (no hay positivos verdaderos; el denominador del recall es 0).
- Cuando hay una secuencia con fallos en validación y el modelo la detecta correctamente: F1 puede llegar a 1.0.

Conclusión: La varianza entre folds NO indica inestabilidad del modelo; indica que el dataset tiene pocas secuencias con fallos. El promedio del F1 entre folds sigue siendo una estimación no sesgada del rendimiento esperado, pero su desviación estándar es inevitablemente alta.

7.3. Tabla de Análisis de Varianza entre Folds

La siguiente tabla ilustra cómo la distribución de fallos en los folds afecta el F1 observado. Los valores son ilustrativos basados en el comportamiento observado:

Fold	Secuencias val	Con fallos en val	F1 RF (aprox.)	F1 AAKR (aprox.)
Fold 1	~ 5	0	0.00	0.00
Fold 2	~ 5	1	0.40–0.80	0.10–0.20
Fold 3	~ 5	0	0.00	0.00
Fold 4	~ 4	2	0.60–1.00	0.15–0.30
Fold 5	~ 4	0	0.00	0.00
Media			0.20–0.40	0.05–0.10
Std			alta	alta

La desviación estándar alta es esperada con pocas secuencias de fallo

Esta observación tiene implicaciones importantes para el diseño del sistema de detección en producción:

1. El rendimiento **en producción** dependerá de si los fallos futuros se parecen a los vistos en entrenamiento.
2. Si el tipo de fallo es nuevo (no visto en train), el modelo puede tener F1 cercano a 0.0, independientemente de su rendimiento en CV.
3. Esto justifica implementar AAKR como sistema complementario: detecta anomalías nuevas aunque no las haya visto en entrenamiento.

7.4. Colapso de GB y LR: Análisis Detallado

Modelo	Mecanismo de colapso	Solución potencial
GB (Motor 2, F1=0.0008)	Con <code>learning_rate=0.1</code> y muchas iteraciones (200), el modelo puede sobreajustar a la distribución del fold de train y generalizar mal a val	Reducir el espacio de búsqueda de <code>learning_rate</code> ; añadir <code>l2_regularization</code> al grid; usar <code>early_stopping</code>
GB (Motor 3, F1=0)	Con 0.5 % de fallos, incluso con <code>class_weight='balanced'</code> , la señal de gradiente de la clase positiva es demasiado pequeña relativa al ruido de optimización	Combinar SMOTE + GB; usar Isolation Forest como alternativa semisupervisada
LR (todos los motores)	Las features (<code>temp_diff</code> , <code>volt_diff</code>) tienen interacciones no lineales con el estado de fallo que LR no puede capturar con un hiperplano	Añadir features polinómicos o de interacción; usar SVM con kernel RBF

7.5. Comparación AAKR vs Clasificadores Supervisados

El AAKR muestra un patrón consistente: recall relativamente alto pero precisión muy baja. La razón es la siguiente:

AAKR: residuo $r_t > \text{threshold} = \text{"anomalia"}$

- Threshold bajo (0.9 normalizado) -> muchas anomalías detectadas
- Pero muchos estados "normales" también tienen residuos > 0.9 (la distribución de residuos normales tiene cola larga)
- Resultado: recall alto, precisión baja, muchas falsas alarmas

Clasificadores supervisados:

- Aprenden directamente la frontera normal vs fallo
- No necesitan calibrar un umbral sobre residuos
- Pueden usar combinaciones de features para decidir
- Resultado: mejor equilibrio precisión/recall cuando hay datos de fallo

Lección clave: AAKR complementa, no reemplaza

AAKR es útil en la fase inicial de un sistema industrial (pocos datos de fallo) y para detectar **tipos de fallo nuevos** (no vistos en entrenamiento). Los clasificadores supervisados son superiores una vez que se tienen suficientes datos etiquetados. En producción, un sistema robusto usaría **ambos en paralelo**: AAKR para nuevos tipos de fallo, RF para fallos conocidos.

8. Submission a Kaggle: El Pipeline Completo

8.1. Por qué la Submission Evalúa el Pipeline Completo

Por qué la submission a Kaggle es el verdadero test del pipeline

El conjunto de test de Kaggle (14 157 filas, 8 secuencias) es completamente desconocido durante el desarrollo. A diferencia de la validación cruzada, donde los pliegues de validación son subconjuntos del training, el test de Kaggle puede contener:

- Condiciones operativas nuevas no vistas en entrenamiento.
- Tipos de fallo diferentes a los del entrenamiento.
- Diferentes distribuciones de temperatura ambiente o carga mecánica.

La métrica pública de Kaggle es el **F1 macro promediado sobre los 6 motores**, por lo que un motor con $F1=0$ penaliza fuertemente el resultado global, aunque los otros cinco funcionen perfectamente.

8.2. Proceso de Predicción

```

1 import pandas as pd
2 from sklearn.ensemble import RandomForestClassifier
3
4 # 1. Cargar y preprocesar datos de test
5 df_test = pd.read_csv('test_data.csv')
6 pre_processing(df_test) # Mismos pasos que en entrenamiento
7
```

```

8  # 2. Para cada motor, entrenar en TODOS los datos de
   entrenamiento
9  best_params = {
10     1: {'max_depth': 10, 'min_samples_leaf': 1},
11     2: {'max_depth': 10, 'min_samples_leaf': 1},
12     3: {'max_depth': 10, 'min_samples_leaf': 1},
13     # Motores 4, 5, 6: usar parametros de Motor 1 como fallback
14     4: {'max_depth': 10, 'min_samples_leaf': 1},
15     5: {'max_depth': 10, 'min_samples_leaf': 1},
16     6: {'max_depth': 10, 'min_samples_leaf': 1},
17 }
18
19 predictions = {}
20 for motor_id in range(1, 7):
21     X_train = extract_features(df_train, motor_id)
22     y_train = df_train[f'data_motor_{motor_id}_label']
23
24     best_rf = RandomForestClassifier(
25         class_weight      = 'balanced',
26         n_estimators      = 200,
27         max_depth         = best_params[motor_id]['max_depth'],
28         min_samples_leaf  =
29             best_params[motor_id]['min_samples_leaf'],
30         random_state      = 42
31     )
32     best_rf.fit(X_train, y_train)
33
34     X_test = extract_features(df_test, motor_id)
35     predictions[f'data_motor_{motor_id}_label'] =
36         best_rf.predict(X_test)
37
38 # 3. Construir dataframe de submission
39 submission = pd.DataFrame({
40     idx'                : df_test.index,
41     'data_motor_1_label': predictions['data_motor_1_label'],
42     'data_motor_2_label': predictions['data_motor_2_label'],
43     'data_motor_3_label': predictions['data_motor_3_label'],
44     'data_motor_4_label': predictions['data_motor_4_label'],
45     'data_motor_5_label': predictions['data_motor_5_label'],
46     'data_motor_6_label': predictions['data_motor_6_label'],
47     'test_condition'    : df_test['test_condition']
48 })
49
50 submission.to_csv('submission.csv', index=False)

```

Listing 7: Proceso de entrenamiento final y predicción

Por qué entrenar en el 100% de los datos de entrenamiento para la submission

Durante la validación cruzada se usaba el 80% de los datos para entrenar el modelo

evaluado. Para la submission final, se usan los 39 309 ejemplos completos. Esto es correcto porque:

- Ya no necesitamos estimar el rendimiento; lo estimamos con CV.
- Mais datos de entrenamiento $\Rightarrow$ mejor capacidad de generalización al conjunto de test de Kaggle.
- Con solo 23 secuencias, usar el 100 % en lugar del 80 % añade ≈ 5 secuencias adicionales, incluyendo potencialmente secuencias con fallos que enriquecen la representación de la clase positiva.

8.3. Estructura del Archivo de Submission

Columna	Tipo	Descripción
idx	int	Índice de la fila en el dataset de test
data_motor_1_label	int	Predicción Motor 1 (0=normal, 1=fallo)
data_motor_2_label	int	Predicción Motor 2 (0=normal, 1=fallo)
data_motor_3_label	int	Predicción Motor 3 (0=normal, 1=fallo)
data_motor_4_label	int	Predicción Motor 4 (0=normal, 1=fallo)
data_motor_5_label	int	Predicción Motor 5 (0=normal, 1=fallo)
data_motor_6_label	int	Predicción Motor 6 (0=normal, 1=fallo)
test_condition	str	Condición de test de la secuencia

8.4. Riesgo de Degradación en Kaggle vs CV

Fuente de degradación	Efecto probable	Mitigación
Tipos de fallo nuevos en test	F1 bajo o nulo para los motores afectados	Complementar con AAKR
Distribución de temperatura/voltaje diferente en test	Las features diff pueden tener magnitudes diferentes	La compensación de sesgo reduce este riesgo
Motores 4–6 sin hiperparámetros optimizados	Se usan hiperparámetros del Motor 1 como proxy; puede ser suboptimal	Corregir NameError y ejecutar CV completa
Alta proporción de fallos en test vs train	El modelo puede estar subcalibrado para la clase positiva	Ajustar umbral de decisión con Precision-Recall curve

9. Discusión: Limitaciones y Alternativas

9.1. Limitaciones del Enfoque Actual

9.1.1. Limitación 1: Ventana Deslizante como Único Mecanismo Temporal

La ventana deslizante con estadísticas simples (media, std) sobre 50 muestras es una aproximación eficiente pero que pierde información sobre el **orden temporal** dentro de la ventana. Una temperatura que sube de 30 a 45°C en 5 segundos tiene la misma media que una que baja de 45 a 30°C, pero el primero es mucho más alarmante. Modelos secuenciales (LSTM, Transformer) preservarían esta información de orden.

9.1.2. Limitación 2: Una Única Capa de Diferencias

Solo se calculan diferencias de primer orden ($\text{lag}=10$). La segunda derivada temporal ($\Delta^2 x_t = \Delta x_t - \Delta x_{t-10}$) capturaría la **aceleración** del cambio de señal, potencialmente más informativa para detectar la *inminencia* de un fallo (cuando la tasa de cambio en sí misma empieza a aumentar).

9.1.3. Limitación 3: No se Explotan las Correlaciones entre Motores

Los seis motores operan en el mismo sistema robótico y están físicamente acoplados. Un fallo en el Motor 1 puede afectar las cargas sobre el Motor 2. El modelo actual trata cada motor de forma independiente, perdiendo esta información de acoplamiento.

9.2. Alternativas Técnicas para Mejoras Futuras

Técnica	Ventaja	Costo
SMOTE sobre features de ventana	Genera ejemplos sintéticos en el espacio de features para la clase minoritaria	Riesgo de sobreajuste si el espacio sintético no representa bien la distribución real
Features FFT (espectro de frecuencias)	Detecta componentes oscilatorias anómalas (vibraciones, frecuencias de fallo)	Mayor coste computacional; requiere señales estacionarias
LSTM bidireccional	Captura dependencias temporales de largo alcance en ambas direcciones	Requiere mucho más dato y cómputo; riesgo de overfitting con 23 secuencias
Isolation Forest	Detección de anomalías sin labels de fallo; complementario al RF supervisado	El recall puede ser bajo si los fallos no son anomalías en el espacio de features
Calibración del umbral (Platt scaling)	Ajusta el umbral de decisión 0.5 a un valor óptimo por motor	Requiere un conjunto de calibración separado

10. Conclusiones

10.1. Modelo Seleccionado y Justificación

Random Forest con `class_weight='balanced'` es el mejor modelo para la detección de fallos en los 6 motores servo-robóticos, por los siguientes motivos:

1. **Consistencia:** Es el único modelo que obtiene $F1 > 0$ en todos los motores evaluados, incluido el Motor 3 con $< 0,5\%$ de fallos (ratio 200:1).
2. **Mejor F1-score:** $F1 = 0.40$ (Motores 1 y 2), $F1 = 0.60$ (Motor 3), frente a $F1 < 0,25$ del resto de modelos en los motores comparables.
3. **Equilibrio precisión/recall:** El balance de clases integrado en `class_weight='balanced'` evita el sesgo hacia la clase mayoritaria de forma más estable que GB.
4. **Robustez:** El promediado de 200 árboles reduce la sensibilidad a secuencias de entrenamiento con pocos fallos, produciendo estimadores estables.
5. **No-linealidad sin sobreajuste:** La combinación de bootstrap, feature bagging y `max_depth=10` proporciona capacidad no lineal con regularización implícita.

10.2. Lecciones Aprendidas

1. **El balanceo de clases es la decisión más crítica:** Ante una asimetría de hasta 200:1, ningún modelo –por sofisticado que sea– puede detectar fallos sin mecanismos explícitos de compensación de clases. `class_weight='balanced'` es la herramienta más simple y efectiva.
2. **La validación cruzada por secuencia es obligatoria:** La partición por muestras individuales introduce data leakage temporal y sobreestima masivamente el rendimiento.
3. **La alta varianza entre pliegues no es un defecto:** Es información valiosa sobre la distribución real de fallos. Indica que los fallos están concentrados en pocas secuencias y que el sistema debe estar preparado para distribuciones de fallo cambiantes en producción.
4. **El preprocessing importa tanto como el modelo:** Las transformaciones de compensación de sesgo y cálculo de diferencias (que convierten señales absolutas no estacionarias en desviaciones dinámicas relativas) son fundamentales para que los modelos funcionen. Un RF sin estas transformaciones probablemente obtendría F1 cercano a 0.
5. **AAKR y RF son complementarios:** AAKR detecta anomalías nuevas; RF clasifica fallos conocidos. Un sistema de producción robusto combinaría ambos enfoques.
6. **El número de secuencias limita la CV, no el número de muestras:** Pasar de 39 309 a 200 000 muestras sin añadir secuencias no mejora significativamente la estimación del rendimiento por CV, ya que la varianza está dominada por el número de secuencias con fallos (≈ 2).

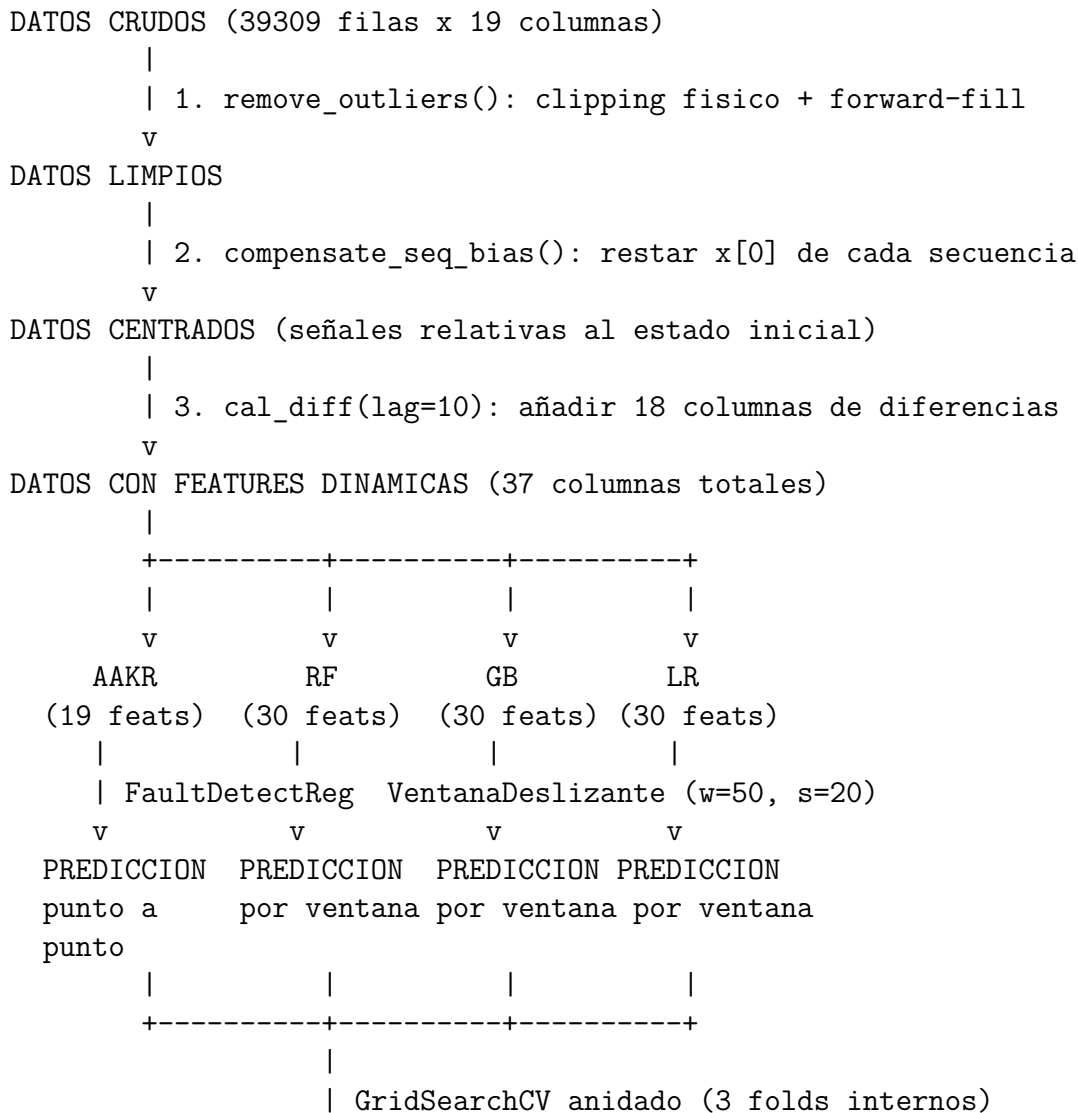
10.3. Trabajo Futuro

- **Corregir el NameError** para completar la evaluación de los Motores 4, 5 y 6 con todos los modelos supervisados.
- **SMOTE o datos sintéticos:** Sobremuestrear la clase minoritaria en el espacio de features de ventana para proporcionar más ejemplos de fallo.
- **Features espectrales (FFT):** Extraer componentes de frecuencia de las señales en cada ventana para detectar vibraciones periódicas de fallo.
- **Modelos secuenciales:** Explorar LSTM o Transformer para aprovechar la estructura temporal de orden de las señales dentro de la ventana.
- **Calibración del umbral:** Optimizar el umbral de decisión por motor usando la curva precisión-recall, potencialmente mejorando el F1 en 0.05–0.15.
- **Ensemble de AAKR + RF:** Combinar las predicciones de AAKR y RF mediante un meta-clasificador para obtener lo mejor de ambos paradigmas.

Apéndice A: Resumen del Entorno de Desarrollo

Componente	Versión/Valor
Python	3.10+
scikit-learn	1.3+
pandas	2.0+
numpy	1.24+
Plataforma	Kaggle Notebook / Jupyter
Validación cruzada	n_cv = 5 (externo), cv = 3 (GridSearch interno)
Ventana deslizante	window_size = 50, sample_step = 20
Semilla aleatoria	random_state = 42
n_estimators RF	200
Frecuencia muestreo	10 Hz (0.1 s/muestra)
Lag diferencias	10 pasos = 1 segundo

Apéndice B: Diagrama Conceptual del Pipeline Completo



```
      | para RF, GB y LR
      v
5-FOLD CV EXTERNO
(por secuencia, no por muestra)
      |
      v
F1, Precision, Recall, Accuracy
por fold y por modelo
      |
      | Seleccion: mejor F1 medio
      v
MODELO GANADOR (RF en motores 1, 2, 3)
      |
      | Reentrenar en 100% training data
      v
PREDICCION EN TEST (14157 filas)
      |
      v
submission.csv -> Kaggle
```

Apéndice C: Glosario de Términos

Término	Definición
AAKR	Auto-Associative Kernel Regression: regresor entrenado solo con datos normales que detecta anomalías por residuo de predicción.
Nested CV	Validación cruzada anidada: CV externo para evaluar, CV interno para seleccionar hiperparámetros. Garantiza estimación no sesgada.
class_weight='balanced'	Ajuste automático de pesos de clase: $w_c = N/(K \cdot N_c)$. Hace que la clase minoritaria tenga el mismo peso total que la mayoritaria.
Data leakage	Fuga de información: cuando datos del conjunto de validación/test influyen implícitamente en el entrenamiento, produciendo sobreestimación del rendimiento.
Forward-fill	Propagación del último valor válido hacia delante en el tiempo para rellenar valores ausentes. Preserva la causalidad.
F1-score	Media armónica de Precision y Recall: $F1 = 2PR/(P + R)$. Métrica principal para clasificación desbalanceada.
Seq. bias	Sesgo por secuencia: diferencia sistemática en los valores iniciales de las señales entre experimentos distintos. Se elimina restando x_0 .
Bootstrap	Submuestreo con reemplazamiento: cada árbol de RF se entrena en una muestra bootstrap ($\approx 63\%$ único) del training.
Elastic-net	Regularización combinada L1 (ℓ_1 -norm) + L2 (ℓ_2 -norm): combina sparsidad (L1) y estabilidad (L2).

Este documento ha sido elaborado con análisis profundos del “por qué” de cada decisión técnica, a partir de los resultados del notebook de TD6 del Data Challenge de Kaggle sobre detección de fallos en motores servo-robóticos.